

Phase 5 Report — LLM-Shaped Reward (RL+LLM Arm)

MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection

Generated 2026-07-09 · Builds on Phase 2 (environment), Phase 3 (RL-only baseline) · Pipeline:
`src/llm/{llm_client,archetypes,reward_shaper}.py` , `src/icsnad_gym/potential.py`

Why this report exists. This closes arm (c) of the 3-way comparison — **LLM-ShAId**, the potential-based, LLM-guided reward shaping layer described in the thesis plan, applied for the first time (per the literature review) to ICS/OT rather than enterprise-IT benchmarks. Everything about the training run is held identical to the Phase 3/4 RL-only baseline except the reward shaping term, so any measured difference is attributable to the LLM's guidance specifically.

1. Executive Summary

Qwen3-14B local LLM, 4-bit (10.6GB VRAM)	25 flow archetypes, k-means	0.909 test F1, RL+LLM (vs 0.845 RL-only)	46% of remaining GBDT gap closed
--	---------------------------------------	--	--

With every other variable held fixed (hyperparameters, seeds, network architecture, episode sampling, 2,000,896 training steps), enabling the LLM-shaped potential-based reward term improved test F1 from 0.845 to **0.909** and recall from 0.734 to **0.845**, while precision stayed comparably high (0.996 → 0.982). Validation performance was essentially unchanged (0.811 → 0.810), meaning the gain shows up specifically in generalization to held-out capture sessions — the exact axis Phase 4 flagged as the hardest one for this dataset.

2. LLM Stack

Component	Choice	Note
Model	Qwen/Qwen3-14B	Plan named Qwen2.5-14B-Instruct with an explicit instruction to check for a newer same-class model at implementation time; Qwen3 is that newer generation (~6 months newer than the plan's knowledge cutoff)
Quantization	bitsandbytes 4-bit (NF4)	Plan's primary choice was AWQ via vLLM/gptqmodel — both hit environment-specific dead ends (see Section 5) before landing on bitsandbytes, which just worked
VRAM footprint	10.6 GB measured	Comfortably within the RTX 4090's 24GB, leaves headroom for concurrent RL training (which itself uses only CPU — see Phase 3)
Inference mode	<code>enable_thinking=False</code>	Qwen3 defaults to an extended reasoning trace before answering; disabled for direct, fast structured-output extraction (generation time: ~10s → ~3s per call)

Calls in RL hot
loop Zero

All 25 archetype risk scores are computed once, offline, and cached — the RL environment only does nearest-centroid lookups (pure NumPy) during training and inference

3. Archetype Clustering & LLM Risk Scoring

150,000 train-split flows (stratified sample) clustered into 25 archetypes via k-means, in the same 55-dimensional feature space (49 numeric + 6 protocol one-hot) as the supervised baselines. Each archetype's centroid statistics were sent to the LLM once, producing a 0–10 risk score and one-sentence rationale, validated against a `{"risk": int, "rationale": str}` pydantic schema with retry-then-fallback on malformed output (never triggered in this run).

LLM risk scores correlate with ground-truth attack rate at $r=0.686$ across the 25 archetypes — a moderate-strong positive correlation, achieved from statistical summaries alone with no access to the ground-truth labels. Archetypes with `attack_rate=1.0` mostly scored 8–10; archetypes with `attack_rate≈0` mostly scored 0–1.

A documented limitation: the archetype description prompt emphasizes TCP flag rates (SYN/ACK/FIN/RST) and fragment rate, which are the most informative features for TCP-based floods and scans but are legitimately near-zero for ICMP/ARP-based attacks (SMURF, FRAGGLE, ARP spoofing). Several high-attack-rate archetypes got correctly scored 8–10 anyway, but via reasoning like "no legitimate TCP session established" rather than identifying the actual ICMP/ARP attack pattern — the right answer for a less precise reason. A future iteration should include protocol-specific descriptive stats (e.g. ICMP type/code distributions) in the prompt rather than a fixed TCP-centric template.

4. Bug Found & Fixed: Softmax Underflow in the Potential Function

The first working version of $\Phi(s)$ returned ≈ 0 for every state regardless of input — verified via direct inspection: distances from a real flow to the 25 archetype centroids ranged 187–353 in the scaled feature space, but the softmax temperature was set to 2.0, so `exp(-dist/2)` computed things like `exp(-96)` for every archetype. All 25 weights underflowed toward the same floating-point noise floor, making the weighted-average risk score numerically meaningless (observed Φ values: 9.5e-35 to 7.3e-6, not the expected 0–10 range) rather than raising an exception.

Root cause: missing the standard softmax numerical-stability trick — subtracting the minimum (or maximum) value before exponentiating. **Fix:** `weights = exp(-(dists - dists.min()) / T)` instead of `exp(-dists / T)`, plus retuning the temperature from 2.0 to 50.0 to match the actual empirical distance scale. After the fix, $\Phi(s)$ values ranged 2.0–5.2 across a real 50-step episode — sensible and stable. This was caught by directly inspecting per-step Φ values against expectations rather than trusting an initially "plausible-looking" small end-to-end reward delta (a shaping-on-vs-shaping-off diff of -1.17 on a -916 base reward looked small but reasonable at first glance; the bug was that the *first* version's diff was -7×10^{-8} , indistinguishable from exact zero, which is what actually triggered the investigation).

5. Environment/Dependency Issues Along the Way

Issue	Resolution
AWQ loading via <code>transformers</code> required <code>gptqmodel</code> , not the expected <code>autoawq</code> (library transition mid-generation)	Installed <code>gptqmodel</code>
<code>gptqmodel</code> import failed on missing <code>torchvision</code> (eager import in a multi-modal model definition unrelated to this text-only use case)	Installed <code>torchvision</code> (incidentally bumped local torch 2.12→2.13; confirmed harmless to the already-running Phase 3 RL process, which keeps its already-loaded libraries in memory)
<code>gptqmodel</code> 's JIT-compiled Marlin CUDA kernels failed to build (<code>nvcc</code> /PyTorch C++ header incompatibility, <code>AutogradState.h</code> compile error)	Abandoned the AWQ/gptqmodel path entirely rather than chase a compiler toolchain fix; switched to bitsandbytes 4-bit quantization of the full-precision model, which ships prebuilt kernels and needs no local compilation
<code>apply_chat_template(..., return_tensors="pt")</code> returns a dict-like object in this <code>transformers</code> version, not a bare tensor	Added <code>return_dict=True</code> and unpacked via <code>model.generate(**inputs)</code>

None of these blocked the final pipeline — each was diagnosed and routed around within the same session. Documented here because a future re-run on a different machine/library-version combination may hit a different subset of these.

6. Full RL Comparison

Model	Config	Val F1	Test F1	Test Precision	Test Recall
RecurrentPPO v1	500k steps, net_arch=64, no shaping	0.950	0.752	0.684	0.835
RecurrentPPO v2	2M steps, net_arch=128, ent_coef=0.01, no shaping	0.811	0.845	0.996	0.734
RecurrentPPO v2 + LLM	identical to v2, use_llm_shaping=True	0.810	0.909	0.982	0.845
GBDT (Phase 4)	supervised, leak-free split	0.894	0.985	0.971	1.000

Clean, attributable result. v2 and v2+LLM differ in exactly one setting — `use_llm_shaping`. Validation F1 is statistically unchanged (0.811 vs 0.810, within noise), which is exactly what a policy-invariant shaping term should produce on data similar to training. The test-set gain (+0.064 F1, +0.111 recall) shows up specifically on genuinely held-out capture sessions — evidence the LLM's archetype-based guidance improved generalization, not just fit to familiar patterns. This closes roughly 46% of the remaining F1 gap to GBDT (0.140 → 0.076) without any architecture change, more training, or hyperparameter retuning versus v2.

Honest caveats: this is a single-seed comparison. The approved plan calls for ≥ 3 seeds (5 preferred) with a paired Wilcoxon signed-rank test before claiming statistical significance — that multi-seed run is the natural next step to firm up this result rather than rely on one run's outcome. It's also still short of GBDT's test F1, consistent with Phase 3/4's honest framing: the RL formulation optimizes an asymmetric, sequential cost function that a static classifier's F1 doesn't capture, and BLOCK/quarantine dynamics plus detection-latency reward remain capabilities no supervised baseline has at all, regardless of its F1.

7. Files Created

Module	Purpose
<code>llm_client.py</code>	Qwen3-14B load/generate wrapper (bitsandbytes 4-bit, thinking disabled)
<code>archetypes.py</code>	K-means clustering of train-split flows into 25 archetypes with descriptive stats
<code>reward_shaper.py</code>	LLM risk-scoring per archetype: prompt template, pydantic validation, disk cache
<code>potential.py</code>	$\Phi(s)$ potential function: nearest-archetype-weighted risk lookup (no LLM calls at runtime)
<code>reward.py</code>	(updated) <code>potential_based_shaping()</code> : $F(s,s') = \gamma\Phi(s') - \Phi(s)$
<code>env.py</code>	(updated) <code>use_llm_shaping</code> constructor flag, default off — Phase 2/3 behavior unaffected unless explicitly enabled

Outputs: `data/archetypes/{archetypes.json, centroids.npy, scaler.joblib, kmeans.joblib, llm_risk_cache.json}` , `experiments/recurrentppo_llm_shaped/final_model.zip` .

8. Next Steps

This closes Phase 5/6's core deliverable — all three arms of the comparison now exist and are measured on the same leak-free split. Natural next steps: (1) multi-seed replication (≥ 3 seeds) of the v2-vs-v2+LLM comparison with a Wilcoxon test for statistical rigor, (2) the detection-latency metric unique to the sequential framing (flows between validated attack onset and first correct ALERT/BLOCK), which no supervised baseline can report at all, and (3) the stretch goals scoped in the original plan (LLM-generated per-alert explanations, synthetic minority-attack augmentation) if time permits.